

1. **JSX is not HTML** — it's a JavaScript syntax that looks like HTML but runs inside JavaScript code.
JSX needs to be compiled — browsers can read HTML directly, but JSX must be converted (by tools like Babel) into JavaScript before running.
In JSX, we use className instead of class because class is a reserved keyword in JavaScript.
JSX can include JavaScript expressions using `{ }`, but HTML cannot.
JSX follows camelCase for attributes (like `onClick`, `tabIndex`), while HTML uses lowercase (`onclick`, `tabindex`).
2. No, browsers can't understand JSX directly — in Vite, JSX is automatically converted into JavaScript by **esbuild**, so you don't need to set up Babel manually.
3. The function must return element it can return only one element so we use fragments , Fragments in JSX let you group multiple elements **without adding an extra HTML tag** — you use them as `<>...</>` or `<React.Fragment>...</React.Fragment>`.
4. In JSX, inline styles are written as a **JavaScript object** using **camelCase** property names `<h1 style={{ color: "blue", fontSize: "20px" }}>Hello</h1>`
5.
 - JSX tags must be **properly closed** (e.g., `<img />`, `<div></div>`).
 - JSX must have **one parent element** wrapping all child elements.
 - **Attributes use camelCase** (e.g., `className`, `onClick`).
 - JavaScript expressions go inside **curly braces** `{ }`.
6. The key attribute in JSX **helps React identify elements uniquely** in lists, so it can efficiently update or re-render only the changed items.